

Project Title : AWS Customer Agreement RAG Assistant

Developer : Harish C

Role : Junior AI Developer Assignment

Introduction

The AWS Customer Agreement RAG Assistant is an AI-powered document question-answering system developed using Retrieval Augmented Generation (RAG). The application enables users to interact with the AWS Customer Agreement document by asking natural language questions and receiving context-aware answers.

The system combines document retrieval, vector similarity search, and Large Language Models (LLMs) to generate accurate responses based only on the provided document content. It uses a local LLM approach with Ollama to ensure efficient and independent AI inference.

The project includes a complete GenAI workflow with a FastAPI backend, Streamlit user interface, FAISS vector database, and SQL-based analytics monitoring system.

Project Overview

Large documents such as cloud service agreements are difficult to manually analyze and traditional keyword search cannot fully understand user intent. This project implements a Retrieval Augmented Generation (RAG) based AI assistant that enables users to ask natural language questions from the AWS Customer Agreement document. The system converts document content into embeddings, stores them in a FAISS vector database, retrieves relevant information, and uses Qwen2.5:3B-Instruct through Ollama to generate accurate context-based answers. The application is developed using LangChain LCEL, FastAPI, SQLite analytics logging, and Streamlit interface with support for source references and query monitoring.

Problem Statement

Organizations often deal with large technical and legal documents containing important information that is time-consuming to search manually. Traditional search methods mainly depend on exact keyword matching and may fail to understand the semantic meaning behind user questions.

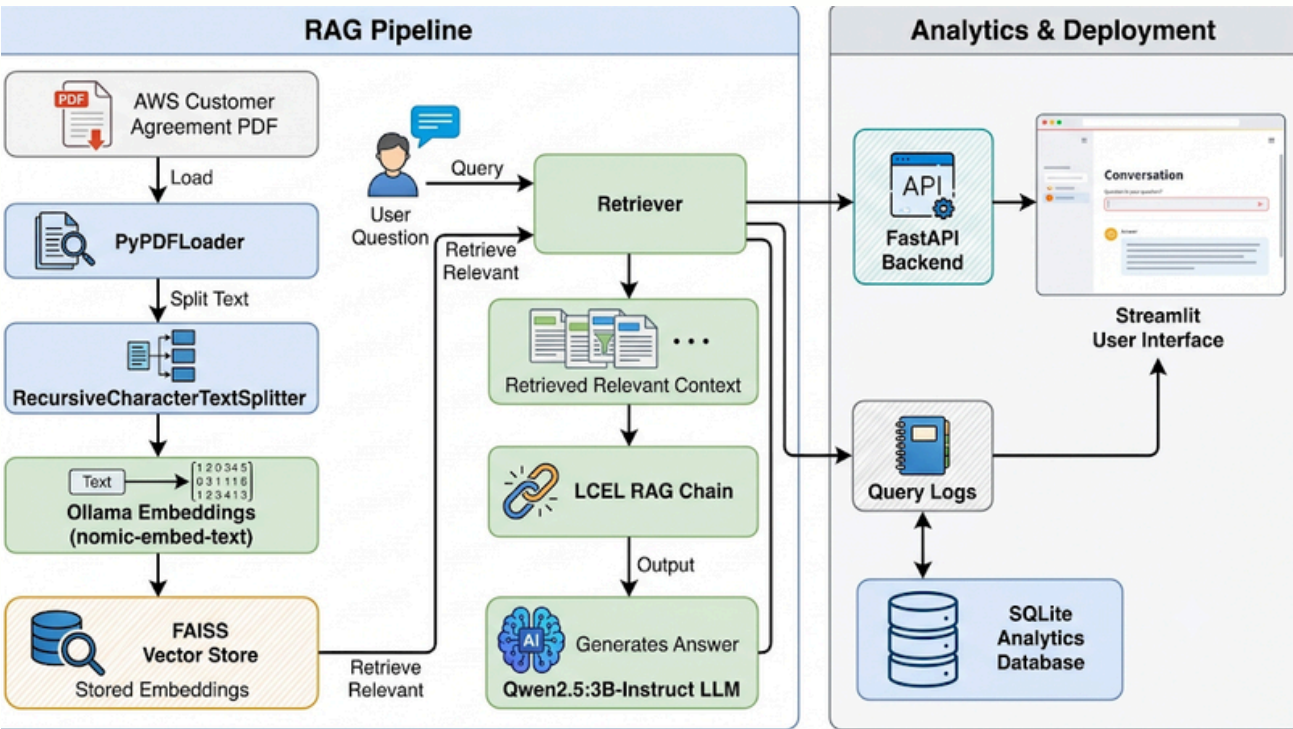
The objective of this project is to develop an AI-powered document assistant that can understand user queries, retrieve relevant information from the AWS Customer Agreement document, and generate accurate answers while minimizing incorrect or unrelated responses from the language model.

Proposed Solution

The proposed solution uses Retrieval Augmented Generation (RAG) to improve document-based question answering. Instead of relying only on the knowledge of a Large Language Model, the system first retrieves relevant information from the AWS Customer Agreement document and provides it as additional context to the model.

The document is processed into smaller chunks, converted into vector embeddings, and stored in a FAISS vector database. When a user asks a question, semantic similarity search identifies the most relevant document sections, which are passed to the Qwen2.5:3B-Instruct model for generating accurate responses. This approach improves answer reliability, reduces hallucinations, and ensures responses are based on the provided document.

System Architecture



Technology Stack

The project uses modern Generative AI frameworks and backend technologies to build an end-to-end document question-answering system.

Programming Language

Python

Used for implementing the backend services and AI workflow.

AI Framework

LangChain + LCEL

Used for building the RAG pipeline and connecting retriever, prompt, LLM, and output parser.

Document Processing

PyPDFLoader & RecursiveCharacterTextSplitter

Used for extracting PDF content and splitting documents into manageable chunks.

Embedding Model

Ollama Embeddings (nomic-embed-text)

Used to convert text chunks into vector embeddings for semantic search.

Vector Database

FAISS

Used for storing document embeddings and performing similarity-based retrieval.

Large Language Model

Qwen2.5:3B-Instruct (Ollama)

Used for generating context-aware answers from retrieved document information.

Backend

FastAPI

Used for creating REST APIs for ingestion, question answering, and analytics.

Database

SQLite + SQLAlchemy

Used for storing query logs, response time, and analytics information.

Frontend

Streamlit

Used for building the user interface and analytics dashboard.

Implementation Workflow

Document Processing:

The AWS Customer Agreement PDF is loaded using PyPDFLoader and split into smaller chunks using RecursiveCharacterTextSplitter with chunk overlap to preserve context.

Embedding & Vector Storage:

Text chunks are converted into vector embeddings using Ollama nomic-embed-text model and stored in FAISS vector database for semantic similarity search.

Retrieval & Generation:

When a user asks a question, FAISS retrieves the most relevant chunks. The retrieved context and question are processed through a LangChain LCEL pipeline and passed to Qwen2.5:3B-Instruct LLM to generate document-based answers.

Backend API Design

The backend is developed using FastAPI to expose REST APIs for communication between the user interface and RAG pipeline.

POST /ingest

Processes the AWS PDF document, generates embeddings, and creates the FAISS vector database.

POST /ask

Accepts user questions, retrieves relevant document chunks, generates answers using the LLM, and returns source references.

GET /analytics

Retrieves stored query logs and provides analytics such as total questions, failed queries, and average response time.

Database Design

The application uses SQLite with SQLAlchemy to store user interaction history and generate analytics.

A query logging table is maintained with the following information:

- Question – User input query
- Answer – Response generated by the RAG system
- Response Time – Time taken to generate the answer

- Status – Success or failed response tracking
- Timestamp – Date and time of query execution

The stored data is used to calculate total queries, average latency, and failed question analytics.

Results

Figure 1: Document Question Answering

Description:

The RAG assistant successfully retrieves relevant information from the AWS Customer Agreement document and generates an accurate answer with source references.

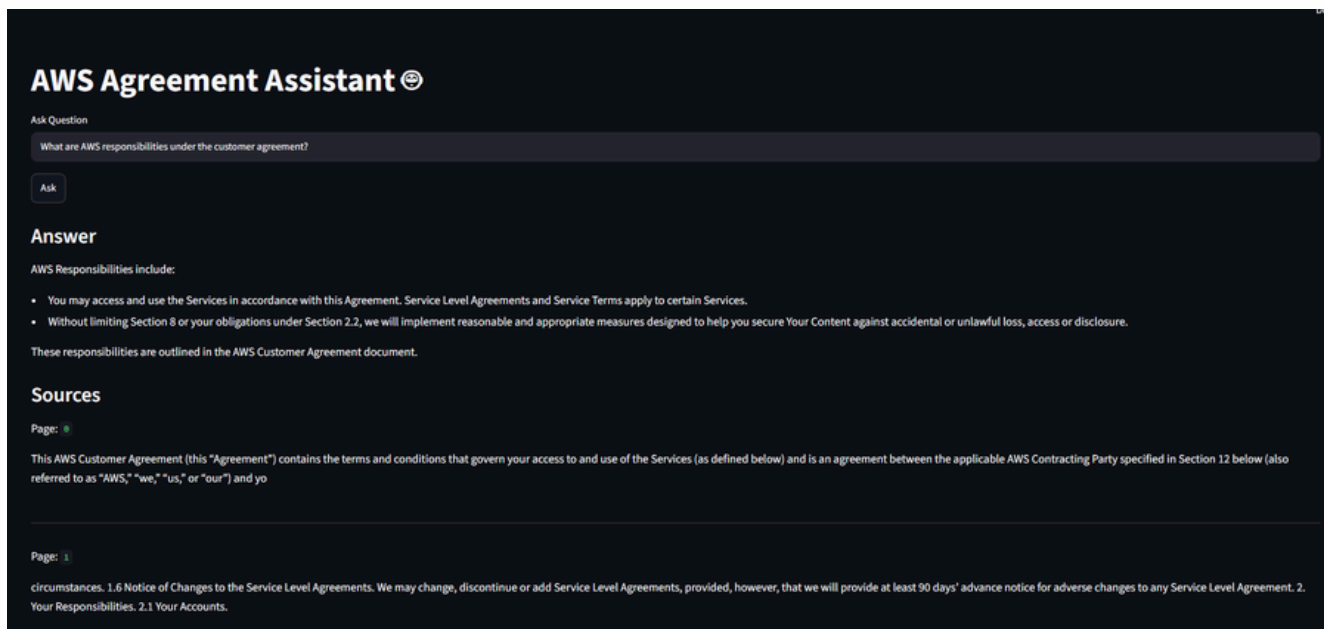


Figure 2: Out-of-Context Query Handling

Description:

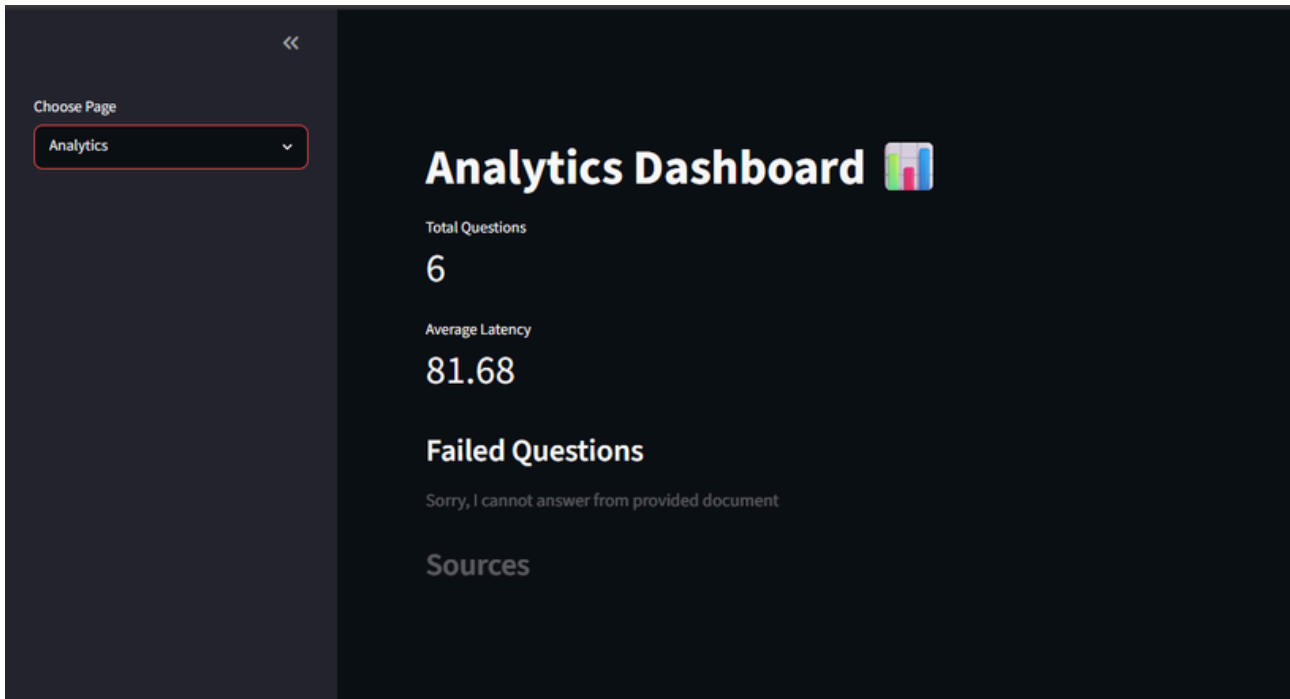
The system identifies questions outside the provided document knowledge and prevents hallucination by returning a fallback response.



Figure 3: Analytics Dashboard

Description:

The analytics dashboard tracks total user queries, average response latency, and failed questions using stored SQLite logs.



Conclusion

The AWS Agreement RAG Assistant successfully implements a Retrieval Augmented Generation based document question-answering system. It enables users to retrieve accurate answers from large PDF documents using semantic search and a local LLM while providing source references and analytics tracking.

References

- [LangChain Documentation](#)
- [FastAPI Documentation](#)
- [FAISS Vector Database](#)
- [Ollama Documentation](#)
- [Streamlit Documentation](#)